Transportation Research Part C 146 (2023) 103946

Contents lists available at ScienceDirect

Transportation Research Part C

journal homepage: www.elsevier.com/locate/trc

Unboxing the graph: Towards interpretable graph neural networks for transport prediction through neural relational inference

Mathias Niemann Tygesen *, Francisco Camara Pereira[1], Filipe Rodrigues[1]

DTU Management Engineering, Transport DTU, Technical University of Denmark, 2800 Kgs. Lyngby, Denmark

ARTICLE INFO

Keywords:
Graph neural networks
Spatio-temporal prediction
Demand prediction
Link based speed prediction
Neural Relational Inference

ABSTRACT

Predicting the supply and demand of transport systems is vital for efficient traffic management, control, optimization, and planning. For example, predicting where from/to and when people intend to travel by taxi can support fleet managers in distributing resources; better predictions of traffic speeds/congestion allows for pro-active control measures or for users to better choose their paths. Making spatio-temporal predictions is known to be a hard task, but recently Graph Neural Networks (GNNs) have been widely applied on non-Euclidean spatial data. However, most GNN models require a predefined graph, and so far, researchers rely on heuristics to generate this graph for the model to use. In this paper, we use Neural Relational Inference to learn the optimal graph for the model. Our approach has several advantages: 1) a Variational Auto Encoder structure allows for the graph to be dynamically determined by the data, potentially changing through time; 2) the encoder structure allows the use of external data in the generation of the graph; 3) it is possible to place Bayesian priors on the generated graphs to encode domain knowledge. We conduct experiments on two datasets, namely the NYC Yellow Taxi and the PEMS-BAY road traffic datasets. In both datasets, we outperform benchmarks and show performance comparable to state-of-the-art. Furthermore, we do an in-depth analysis of the learned graphs, providing insights on what kinds of connections GNNs use for spatio-temporal predictions in the transport domain and how these connections can help interpretability.

1. Introduction

A significant factor in modern intelligent transport systems is preemptively adapting to changes. This could be changes in taxi demand in a city or speeds on a road network. By matching the actual taxi demand, planners can best utilize the fleet of taxis, both lowering costs and waiting times for users. However, doing so requires a good demand model that allows transport planners to best plan the supply and adopt early interventions when the demand suddenly changes. However, ride-hailing transport systems' demand prediction has highly non-euclidean spatial dependencies between zones. There could be dependencies between residential and industrial zones, between zones with many hotels and zones with many tourist attractions, modal interchange stations and other areas, etc. Similarly, for road networks, vehicles can be rerouted around those areas to optimize traffic flow better by predicting traffic jams.

In recent years, Graph Neural Networks (GNNs) have shown great performance on different tasks in non-euclidean spaces (Bronstein et al., 2021). Also, in transport systems, GNNs have shown remarkable performance in both link-based flow/speed prediction

* Corresponding author.
E-mail address: mnity@dtu.dk (M.N. Tygesen).
[1] These authors contributed equally.

https://doi.org/10.1016/j.trc.2022.103946
Received 25 January 2022; Received in revised form 19 September 2022; Accepted 23 October 2022
Available online 24 November 2022

and demand prediction for different transport systems (Li et al., 2018; Ye et al., 2021). While the road network is the inherent graph for many flow/speed prediction tasks, this graph may not be optimal as a road link could have higher-order dependencies with road links further away. Furthermore, creating the true graph for a road network might require error-prone map-matching procedures. For ride-hailing systems, there is no inherent graph, as the system is divided into zones or stations with no natural connections. To address this, researchers have previously used different heuristic methods to create a graph of the system for the GNNs to use. While these heuristic graphs allow for application of domain knowledge, they are a modeling choice and might not depict the true correlation or dependencies between roads, zones, or stations. On top of the spatial dependencies, transport systems also have strong temporal dependencies with daily and weekly trends in demand. Previously, this has been modeled by including Recurrent Neural Networks to take these trends into account on a per-node basis. However, heuristically-created graphs are static through time which means that models have the same assumption regarding correlations between zones for all time steps. Unfortunately, the true dependencies in a transport system might change over time. For example, it seems highly plausible that the dependencies between a residential zone and an industrial zone with many jobs show clear dependencies to the industrial zone in the morning and back to the residential zone in the afternoon, but none during weekends or nights.

In this paper, we utilize Neural Relational Inference (NRI) (Kipf et al., 2018), which uses a Variational Auto-Encoder (VAE) structure to learn a distribution over the possible edges in the graph. This way, the model infers the graph that leads to the best predictions. Under the assumption that the true dependency graph will lead to the most accurate predictions, this inferred graph will approximate the true dependencies. Another benefit of this approach is that it enables the researcher to put a Bayesian prior over the latent graph distribution, thus allowing them to apply domain knowledge without limiting the model's flexibility. Furthermore, unlike others, our model is not constrained to a specific number of nodes. Hence, the model can be used in settings where the number of nodes is different throughout the dataset, i.e., a road network with a varying number of loop detectors over a few years. Therefore, the model could also be used in a transfer learning setup where the model is trained on a transport system with a large amount of data and later fine-tuned to a scenario with sparse data, even if the number of nodes is different.

Another benefit of using the inferred graph is that we get potential insights into what information the models deem important. In other words, the inferred graph allows us to, for each specific data point, see what kind of connections between nodes the model has deemed important for its predictions. As such, following the definitions of Molnar (2022), this can be seen as intrinsic, model-specific, local, and modular interpretability. However, this only has value if the gained insights are stable over multiple predictions, are comprehensible enough for humans to understand, and are translucent enough that they can reliably be found. Therefore, on top of conducting experiments that show the model's predictive capabilities, we also analyze the inferred graphs, showing the types of connections the model prefers in different transport systems showing the potential for interpretability and its limitations. To the best of the authors' knowledge, this is the first in-depth analysis of inferred graphs in transport data. We do this to demonstrate that the learnable adjacency matrix is a step towards more interpretable models and to show what kind of connections our proposed model learns to optimize predictions.

In summary, our contributions are:

- a proposal for automatic inference of a dynamic graph adjacency matrix for use in a GNN for transport data that shows performance competitive (or, in some cases, superior) with other state-of-the-art GNN approaches;
- a study demonstrating significant performance improvements in comparison with the most common heuristics in GNN graph structure design;
- empirical applications of the method on two transport prediction tasks — ride hailing demand prediction and link speed prediction.
- an analyses of the inferred graphs showing how they can lead to insights in what the model focuses on when creating predictions and a discussion of the limitations of the insights.

2. Literature review

In this section we review deep learning transport research and particularly focus on the graph inference problem in using GNNs on transport data.

2.1. Deep learning in transport research

Prediction tasks in transport systems have seen much research, and many different methods have been used on the problem. Historically, classical statistical methods like ARIMA (Williams and Hoel, 2003) and classical machine learning methods like Gaussian Processes (GP) (Diao et al., 2019) have been used, however these face important challenges when considering large scale urban network spatial and temporal dependencies. Variants exist, like Vector ARIMA (VARIMA Barcelo-Vidal et al., 2011) or Multi-output GPs (Rodriguez-Deniz et al., 2017) that aim to cover such dependencies, but they are either too rigid (once calibrated, parameters are fixed) for real-world cities, or computationally challenging (Liu et al., 2020). However, recent developments in deep learning paved the way for efficient and flexible modeling of spatial and temporal dependencies in transport data. For the temporal dependencies, Recurrent Neural Networks (RNN) have been utilized (Xu et al., 2018). Initially, the spatial dependencies have been modeled using Convolutional Neural Networks (CNN) (Zhang et al., 2016), yet CNN's require the data to follow a grid data structure which not all transport data lends itself to. Therefore, researchers have recently focused on utilizing graph neural networks (GNN) in transport contexts (Li et al., 2018; Jin et al., 2020; Zhang et al., 2019). GNNs' ability to capture both short- and long-distance spatial relationships makes them especially attractive for transport applications as evident by the large amount of research published. See Jiang and Luo (2021) and Yin et al. (2021) for more exhaustive surveys.

2.2. Graph inference in transport research

Most GNN models require a predefined graph. For problems with an intrinsic graph structure like road networks, molecules, or social networks, there exists a natural choice of graph for the GNN to use. However, such graph may not be optimal for accurate predictions. For problems with no intrinsic structure like station or zone-based ride-hailing demand prediction, researchers have no obvious graph for the GNN and have to resort to heuristics to create a graph. Examples of such heuristics include graphs based on geospatial distance, neighboring zones, time-series correlation, or zone functional similarity (e.g., based on points-of-interest, land-use). Even after the choice of metric to use, the researcher also has a choice of how to translate the similarities between nodes into a graph, i.e., choose a cut-off similarity to determine what is an edge and what is not. While this allows applying domain knowledge in the modeling process, these graphs are fixed and might not reflect the true underlying dependencies in the data, leading to bias in the model and lower predictive performance. Multiple approaches have been applied in transport research to try and amend this problem. In Bai et al. (2020) and Wu et al. (2019), the authors randomly initialize one or two matrices of learnable node embeddings and multiply them together, followed by an activation function and softmax to get an adjacency matrix. This adjacency matrix can then be learned during training using stochastic gradient descent. In Tang et al. (2020), the authors calculate attention weights between all nodes in the network and use those as weights in the adjacency matrix. Ye et al. (2021) start with a similarity-based adjacency matrix but use a stack of learnable mapping functions to generate multiple adjacency matrices. The authors then aggregate the embeddings from all the graphs in the final output.

Our method varies from previous methods used in transport research in multiple ways. The VAE structure infers a graph by looking at recent history. This enables the graph to be dynamic throughout time and capture the changing dynamics in the transport system. Furthermore, the inferred graph can be used to see where the model focuses when making predictions given the current state of the system, which in turn can be used to gain insights about how the model leverages spatial correlations, thus making it less of a black-box. The VAE structure also means that the learned encoder is not locked to a specific graph size. This means that a trained model can be utilized when the graph changes size, e.g. if more sensors are added to a highway network or a road is closed due to construction. The encoder and decoder structures also utilize the Graph Network (GN) structure from Battaglia et al. (2018) which allows for both the encoder and the decoder to utilize global features.

3. Definitions

3.1. Graph definition

A graph is defined as a 3-tuple $G = (\boldsymbol{u}, V, E)$. $V = \{\boldsymbol{v}_i\}_{i=1:N^v}$ is the set of N^v vertices and $\boldsymbol{v}_i$ are the ith nodes features. $E = \{(\boldsymbol{e}_k, r_k, s_k)\}_{k=1:N^e}$ is the set of N^e edges were $\boldsymbol{e}_k$ are the edge features and r_k is the receiver node index and s_k is the sender node index. Note that for brevity we will use $\boldsymbol{e}_{(i,j)}$ to refer to $\boldsymbol{e}_k$ where $s_k = i$ and $r_k = j$. $\boldsymbol{u}$ is a global feature for the entire graph, e.g. air temperature for a traffic system. Often the connections of the graph are represented in an adjacency matrix $A \in \mathbb{R}^{n\times n}$ where $A_{ij} = 1$ if $\boldsymbol{e}_{(i,j)} \in E$ and 0 otherwise. Note here that the adjacency matrix notation does not contain the edge features but only the connections.

3.2. Graph neural networks

Graph neural networks have seen much research in recent years. Therefore there are many types of GNNs (Kipf and Welling, 2016; Hamilton et al., 2017; Gilmer et al., 2017). The Graph Network framework (Battaglia et al., 2018) unifies most of these networks. In the GN framework, a GNN consists of GN blocks, each consisting of 3 update functions and three aggregation functions

$$\begin{aligned} \mathbf{e}'_k &= \phi^e\left(\mathbf{e}_k, \mathbf{v}_{r_k}, \mathbf{v}_{s_k}, \mathbf{u}\right) & \bar{\mathbf{e}}'_i &= \rho^{e\to v}\left(E'_i\right) \\ \mathbf{v}'_i &= \phi^v\left(\bar{\mathbf{e}}'_i, \mathbf{v}_i, \mathbf{u}\right) & \bar{\mathbf{e}}' &= \rho^{e\to u}\left(E'\right) \\ \mathbf{u}' &= \phi^u\left(\bar{\mathbf{e}}', \bar{\mathbf{v}}', \mathbf{u}\right) & \bar{\mathbf{v}}' &= \rho^{v\to u}\left(V'\right), \end{aligned} \tag{1}$$

where $E'_i = \left\{\left(\mathbf{e}'_k, r_k, s_k\right)\right\}_{r_k=i,\ k=1:N^e}$, $V' = \left\{\mathbf{v}'_i\right\}_{i=1:N^v}$ and $E' = \bigcup_i E'_i = \left\{\left(\mathbf{e}'_k, r_k, s_k\right)\right\}_{k=1:N^e}$. The edge and node update functions ϕ^e and ϕ^v are mapped across all edges and nodes respectively and the global update ϕ^u function is applied once. ρ is a permutation invariant aggregation function that can take varying number inputs, e.g. elementwise summation, mean or maximum.

3.3. Spatio-temporal prediction in transport systems

The task of spatio-temporal prediction in transport systems is to predict the system's future states, given the historical states. In order for GNN's to be used for the prediction, the system needs to be in a form with nodes. For systems with clear nodes, i.e., stations in a metro network or loop sensors along a highway, this step is trivial. However, for problems with no clear nodes, like a ride-hailing service, the data has to be preprocessed into a node-based form first. This can be done by using predefined zones and cluster the observations into these zones. Another method is using clustering algorithms to cluster the observations into "virtual" zones (Ye et al., 2021). Here we will only use data either in a node form or clustered into predefined zones by the data provider.

For a transport system with N nodes let $\boldsymbol{x}^t \in \mathbb{R}^{N\times c}$ denote the feature matrix at time step t. Here c denotes the feature dimension. Then the task of spatio-temporal prediction is given the last P timesteps, predict the next Q time steps can be denoted as

$$\boldsymbol{x}^{t-P+1:t} \xrightarrow{\mathcal{F}} \boldsymbol{x}^{t+1:t+Q}, \tag{2}$$

where $\mathcal{F}$ is a mapping function, $\boldsymbol{x}^{t-P+1:t} \in \mathbb{R}^{P\times N\times c}$ and $\boldsymbol{x}^{t+1:t+Q} \in \mathbb{R}^{Q\times N\times c}$.

However, as discussed earlier, transport systems show clear spatial correlations that change through time and separately treating each node cannot account for these. Therefore, graph neural networks also utilize a graph structure connecting the nodes. Given that $\mathcal{F}$ is a GN following the structure from Eq. (1), taking in general graph features, the formulation becomes

$$[\boldsymbol{x}^{t-P+1:t}, \boldsymbol{u}^{t-P+1:t}, G] \xrightarrow{\mathcal{F}_{GN}} \boldsymbol{x}^{t+1:t+Q}, \tag{3}$$

where G is the graph structure and $\boldsymbol{u}^t \in \mathbb{R}^{c_u}$ is the general graph features, with c_u dimensions, of time step t, hence $\boldsymbol{u}^{t-P+1:t} \in \mathbb{R}^{P\times c_u}$. Here the graph structure dictates which nodes share information. Therefore having a graph structure that represents the true underlying spatial correlations at a given time will lead to better predictive performance. Hence in a setting with a learned dynamic graph structure, the learned graph structure can be interpreted as an approximation of the true time-varying spatial correlations in the system.

4. Recurrent spatio-temporal graph neural network with neural relational inference

In this section, we present the Neural Relational Inference GNN model. The model follows the basic structure from Kipf et al. (2018) but inspired by Battaglia et al. (2018) a global graph feature is added. This global value is then used to condition the edge-to-node and node-to-edge transformations in both the encoder GNN and the decoder GNN. Furthermore, we fully utilize the prior by constructing a structured prior using domain knowledge. The model can be seen as a VAE (Kingma and Welling, 2014). With our extensions, the model maximizes the Evidence Lower Bound (ELBO):

$$\mathcal{L} = \mathbb{E}_{q_\phi(\boldsymbol{z}|\boldsymbol{x}^{t-P+1:t},\boldsymbol{u})}[\log p_\theta(\boldsymbol{x}|\boldsymbol{z},\boldsymbol{u})] - \mathrm{KL}[q_\phi(\boldsymbol{z}|\boldsymbol{x}^{t-P+1:t},\boldsymbol{u})\|p(\boldsymbol{z})], \tag{4}$$

where

$$p_\theta(\boldsymbol{x}|\boldsymbol{z},\boldsymbol{u}) = \prod_{t'=t+1}^{t+Q} p_\theta(\boldsymbol{x}^{t'+1}|\boldsymbol{x}^{t'},\dots,\boldsymbol{x}^{t+1},\boldsymbol{z},\boldsymbol{u}) \tag{5}$$

is the decoder that predicts the next step given the previous observations, the latent graph structure, and the global feature, and

$$p(\boldsymbol{z}) = \prod_{i\neq j} p(\boldsymbol{z}_{(i,j)}) \tag{6}$$

is a factorized prior on the latent graph. $p(\boldsymbol{z}_{(i,j)})$ can either be the same prior for all the edges or be structured with a different distribution for each edge based on prior knowledge. In a transport setting this domain knowledge could for example be the road network connecting loop detectors or spatial distance between zones. The last term in Eq. (4) then penalizes learned latent graphs that diverge from the prior. Thereby the model will prefer graphs that adhere to the domain knowledge unless this will lead to worse predictive performance.

4.1. Encoder

The encoder $q_\phi(\boldsymbol{z}|\boldsymbol{x})$ takes as input the historical data $\boldsymbol{x}^{t-P+1:t}$ and global feature $\boldsymbol{u}$. The encoder follows the structure of a GNN running on a fully connected graph $\mathbf{1}^{N\times N}$. In the GN framework, the encoder can be seen as three GN blocks stacked after each other: an independent recurrent block that updates the embedding of the nodes and global feature, a block that creates edge features and updates node features, and finally a block that updates the edge features. After that, a fully connected neural network takes the final edge embeddings and outputs the probabilities for the categorical distribution in the latent space. Let $\boldsymbol{x}_j$ denote the features of the jth node, then the node and edgewise computations can be written as

$$\begin{aligned}
\boldsymbol{h}_j^{(1)} &= f_{emb}(\boldsymbol{x}_j^{t-P+1:t}) \\
\boldsymbol{u}^{(1)} &= f_u(\boldsymbol{u}) \\
v \to e: \boldsymbol{h}_{(i,j)}^{(1)} &= f_e^{(1)}([\boldsymbol{h}_i^{(1)}, \boldsymbol{h}_j^{(1)}, \boldsymbol{u}^{(1)}]) \\
e \to v: \boldsymbol{h}_j^{(2)} &= f_v^{(1)}([\sum_{i\neq j} \boldsymbol{h}_{(i,j)}, \boldsymbol{u}^{(1)}]) \\
v \to e: \boldsymbol{h}_{(i,j)}^{(2)} &= f_e^{(2)}([\boldsymbol{h}_i^{(2)}, \boldsymbol{h}_j^{(2)}, \boldsymbol{h}_{(i,j)}^{(1)}]) \\
\boldsymbol{h}_{(i,j)}^{(3)} &= f_p(\boldsymbol{h}_{(i,j)}^{(2)}),
\end{aligned} \tag{7}$$

where the f's are fully connected neural networks. The structure of the encoder is depicted in Fig. 1.

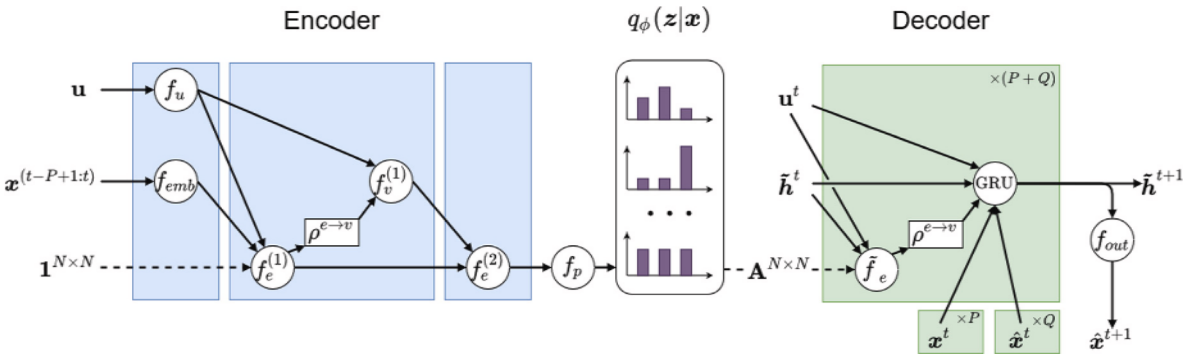

Fig. 1. The full model structure. First, the encoder takes in the global feature as well as the historical data and outputs parameters for the latent distribution $q_\phi(\boldsymbol{z}|\boldsymbol{x})$. Then the concrete distribution is used to get a sample from the latent space. This sampled graph is then used in the recurrent decoder. The decoder takes in the true data $\boldsymbol{x}^t$ for the first P steps and then the last prediction $\hat{\boldsymbol{x}}^t$ for the last Q steps.

4.2. Latent space

In order to be able to backpropagate through the latent space $q_\phi(\boldsymbol{z}_{ij}|\boldsymbol{x})$, we do a continuous relaxation using the Gumbel distribution (Jang et al., 2017; Maddison et al., 2017). This way, the samples are drawn as:

$$\boldsymbol{z}_{(i,j)} = \text{softmax}((\boldsymbol{h}^{(3)}_{(i,j)} + \boldsymbol{g})/\tau), \tag{8}$$

where g is a vector of k i.i.d samples from a Gumbel(0,1) distribution and τ is a temperature that controls the "smooth" relaxation of the samples. We collect these samples into an adjacency matrix $\mathbf{A}^{N\times N}$ s.t. $\mathbf{A}^{N\times N}_{(i,j)} = \mathbf{z}_{(i,j)}$.

4.3. Decoder

In order for the decoder to be able to model the temporal dependencies in $p_\theta(\boldsymbol{x}^{t'+1}|\boldsymbol{x}^{t'}, \dots, \boldsymbol{x}^{t+1}, \boldsymbol{z}, \boldsymbol{u})$ we utilize a Gated Recurrent Unit (GRU) cell in the decoder. The decoder can be seen as a recurrent GN block where the hidden state in the GRU cell is the node embeddings. The block first does an edge embedding using the hidden state and the graph $\mathbf{A}^{N\times N}$ sampled from the latent space. Then, the edge embeddings are used to do a node embedding update using the GRU cell. The GRU also takes the observed node features and global features as input and outputs a hidden state for the next time step. This hidden state is then both sent to the next iteration and used to create predictions for the next time step.

$$\begin{aligned}
v \to e : \tilde{\boldsymbol{h}}^t_{(i,j)} &= \sum_k z_{ij,k} \tilde{f}_e^k([\tilde{\boldsymbol{h}}^t_i, \tilde{\boldsymbol{h}}^t_j, \boldsymbol{u}^t]) \\
e \to v : \text{MSG}^t_j &= \sum_{(i\neq j)} \tilde{\boldsymbol{h}}^t_{(i,j)} \\
\tilde{\boldsymbol{h}}^{t+1}_j &= \text{GRU}([\text{MSG}^t_j, \boldsymbol{x}^t_j, \boldsymbol{u}^t], \tilde{\boldsymbol{h}}^t_j) \\
\boldsymbol{\mu}^{t+1} &= \boldsymbol{x}^t_j + f_{out}(\tilde{\boldsymbol{h}}^{t+1}_j) \\
p(\boldsymbol{x}^{t+1}|\boldsymbol{x}^t, \boldsymbol{z}) &= \mathcal{N}(\boldsymbol{\mu}^{t+1}, \sigma^2 I).
\end{aligned} \tag{9}$$

The tilde is used to denote the decoders hidden states and edge embedding networks such that they can be distinguished from the encoders.

4.4. Training

During training, we first send the historical data $\boldsymbol{x}^{t-P+1:t}$ and the global features $\boldsymbol{u}$ into the encoder. Note that we provide the entire global feature as input to the model. We do this as we will use global features with external predictions that we assume to be of high enough quality to work with the model. However, it is easy to constrain the encoder to only historical global features, i.e., $\boldsymbol{u}^{t-P+1:t}$. The encoder then outputs the latent distribution over graph edges $q_\phi(\boldsymbol{z}|\boldsymbol{x}, \boldsymbol{u})$, from which we sample a graph. We then use the sampled graph for the decoder. The decoder runs $P+Q$ iterations. For the first P iterations, we input the true observations $\boldsymbol{x}^t$ into the decoder, and for the last Q iterations, we input the predictions $\hat{\boldsymbol{x}}^t$ into the decoder.

The loss then consists of a KL divergence part and a reconstruction part. The KL divergence part is given by

$$\text{KL}[q_\phi(\boldsymbol{z}|\boldsymbol{x}, \boldsymbol{u}) \| p(\boldsymbol{z})] = \sum_z q_\phi(\boldsymbol{z}|\boldsymbol{x}, \boldsymbol{u}) \log\left(\frac{q_\phi(\boldsymbol{z}|\boldsymbol{x}, \boldsymbol{u})}{p(\boldsymbol{z})}\right), \tag{10}$$

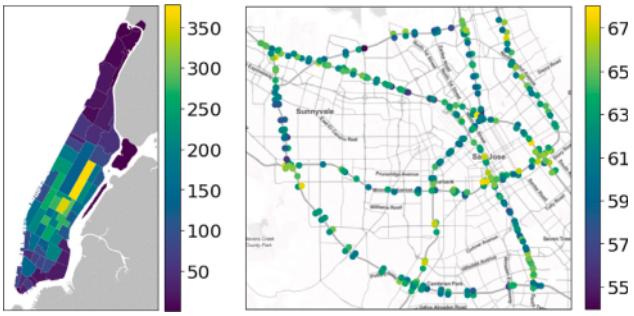

Fig. 2. Left shows the 66 taxi zones of Manhattan color-coded based on the mean hourly activity over the data period. Right shows the location of the 325 loop sensors in the PEMS data. The color is the mean speed at the sensor over the data period. The location of the sensors has been altered slightly to show sensors placed on opposite sides of the same freeway. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

and the reconstruction part is given by Negative Log-Likelihood (NLL) summed over nodes and prediction time steps:

$$\log p_\theta(\boldsymbol{x}|\boldsymbol{z},\boldsymbol{u}) = -\sum_{j}^{N}\sum_{t'=t}^{t+Q}\left(\frac{\|\boldsymbol{x}_j^{t'} - \hat{\boldsymbol{x}}_j^{t'}\|^2}{2\sigma^2} + \frac{1}{2}\log(2\pi\sigma)\right). \tag{11}$$

The value of σ can be seen as a hyperparameter determining how much the negative log-likelihood should penalize errors. This can be used to scale the ratio between the negative log-likelihood and the KL divergence, thereby putting more or less emphasis on the prior.

5. Experiments

In this section, we perform experiments on two different datasets. The source code for all experiments is available.[2]

5.1. Datasets

The experiments are done on two open-source datasets.

- **NYC Yellow Taxi Data**[3]: The dataset contains logs for all of the yellow taxis driving around New York. January to October of 2019 is used, and we extract the data from Manhattan. The data contains 84 million raw trips. Each trip logs the starting zone, ending zone, time of the start and end, trip distance, trip duration, and trip cost. A map showing the mean activity of each zone can be seen in the left part of Fig. 2.
- **PEMS-BAY**[4]: The dataset contains logs of traffic speeds from 325 loop sensors around the Bay Area collected by the California Transportation Agencies (Cal-Trans) from January 1st, 2017 to May 31th 2017. A map showing the mean traffic speed of the sensors can be seen in the right part of Fig. 2

5.2. Data preprocessing

The NYC Yellow Taxi dataset is first cleaned for erroneous logs. We remove all trips with:

- A distance shorter than 0.1 miles;
- A duration shorter than 1 min;
- Free trips or trips with a negative cost;
- Trips with errors in the timestamps.

After cleaning, the dataset contains 71 million trips. We then aggregate the data into pickups and dropoffs in each NYC taxi zone per hour. We create splits of 60 h with 48 h for burn-in and 12 h for prediction, i.e., given the last two days, the task is to predict the pickups and dropoffs in each zone for the next 12 h. We then use a 0.8/0.1/0.1 train, validation, and test split and normalize based on the data in the train set.

The PEMS dataset is taken from Li et al. (2018) and hence follows the same train, validation and test splits and is split into vectors of 24 h with 12 h for burn-in and 12 h for prediction. We also standardize the data before training.

[2] https://github.com/MathiasNT/NRI_for_Transport.
[3] https://www1.nyc.gov/site/tlc/about/tlc-trip-record-data.page.
[4] https://github.com/liyaguang/DCRNN.

Table 1
The table shows the performance of our models and CGC on the NYC Yellow Taxicab data. Our fixed models use different kinds of fixed adjacency matrices and the NRI models use learned dynamic adjacency matrices with different priors.

Model	MAE	RMSE	MAPE	PCC
CGC (Ye et al., 2021)	20.18	33.82	48.0%	0.974
Fixed empty	23.31	49.37	75.0%	0.943
Fixed full	19.41	32.47	43.4%	0.976
Fixed local	22.80	37.91	50.0%	0.967
Fixed DTW	23.21	38.65	49.3%	0.966
NRI local	18.36	31.69	36.8%	0.978
NRI unif	17.54	31.69	36.8%	0.977
NRI DTW	17.65	30.44	33.2%	0.978

5.3. Experiments on the NYC Yellow Taxicab dataset

In this section, we do experiments on the NYC Yellow Taxicab dataset. First, we compare different versions of our model and a baseline from a recent paper. Our models consists of versions using different fixed adjacency matrices and full NRI models with learnable, dynamic adjacency matrices. The full models are trained using different priors. Then, we analyze the learned adjacency matrices, using the best performing model, to see what kind of relations the learned adjacency matrices encode. We do this analysis by first looking into the general temporal trend of the connections, showing how the model is able to catch weekly trends. Afterwards, we look at the general spatial trend of the connections, showing how the model prioritizes sending messages to some zones over others. And lastly we show a more specific spatial trend that shows how the model is able to recognize different types of zones and share information between them.

Performance on NYC Yellow Taxicab dataset

We run experiments on the New York City Yellow Taxi Cab dataset, comparing the inferred adjacency matrix against commonly used adjacency matrices based on heuristics as well as CGC (Ye et al., 2021) which uses a learned but fixed adjacency matrix. The baseline heuristic adjacency matrices are:

- **Local**. Each zone is connected only to its neighboring zones. Two zones are defined as neighbors if they share a border.
- **DTW**. Each zone is connected only to the zones that have the most similar time series. The similarity between each zone is computed using Dynamic Time Warping (DTW) on its time series in the training dataset. The adjacency matrix is then quantized by the 90th percentile to get a binary adjacency matrix.
- **Empty**. An empty adjacency matrix, meaning that the zones do not share information. The model then becomes an RNN looking at each zone separately.
- **Full**. A full adjacency matrix, meaning that all zones are connected to all other zones.

For the models with a fixed adjacency matrix, we use only the decoder part of the model as seen in Fig. 1. Then instead of sampling a new graph from $q_\phi(\boldsymbol{z}|\boldsymbol{x})$, we input the fixed adjacency matrix for all steps. For the full NRI models, we use the full model with three different priors: the local and DTW adjacency matrices as explained above and a uniform prior that puts a prior of 90% probability of no edge on all possible edges. For the NYC Yellow Taxicab dataset, we found a random initialization of the encoder to be too noisy and led to bad training of the model. Therefore, we pre-train the encoder for 30 epochs using only the KL divergence part of the loss. This means that when the model starts full training, the initial outputs of the encoder are similar to the prior.

First, we compare the model with a fixed encoder, the NRI model with different priors on the learned adjacency matrix and the CGC baseline. For the fixed model, the different adjacency matrices are the local adjacency and the DTW adjacency matrices explained above, along with a full and empty adjacency matrix. The results can be seen in Table 1. As can be seen in the table, among the fixed models, the full adjacency matrix achieves the best performance while the empty adjacency matrix achieves the worst performance. This shows that the model benefits from sending information among the nodes. For the heuristic-based graphs, the local graph achieves the best performance, while the DTW based graph significantly outperforms the empty graph on all metrics except for MAE. We also see that CGC outperforms all of the fixed encoder models except the model with a full adjacency matrix. Looking at the full NRI models, we can see that all models outperform CGC and the fixed encoder approaches, including those with a full adjacency matrix. This suggests that the model is able to learn which edges are important and send more specific messages along these edges. We can also see that the model with a uniform prior achieves the best MAE score while the model with the DTW prior achieves the best performance in all the other metrics. All of the models have been trained with a σ value in the loss (Eq. (11)), that allows the model to overrule the prior. As such, the prior works more as a guide for the training of the model. This is especially true when using the pretraining for the encoder, as the initial adjacency matrix will then match the prior as well.

Next, we look at the predictions made by the best model, the NRI DTW. As can be seen in Fig. 3, the model can capture the daily trends. This is especially clear in the Lenox Hill West zone, where each weekday clearly consists of two rush hour peaks. Furthermore, the model is also able to capture weekly trends, as can be seen in the East Village zone, where there is a huge pickup peak during weekend evenings.

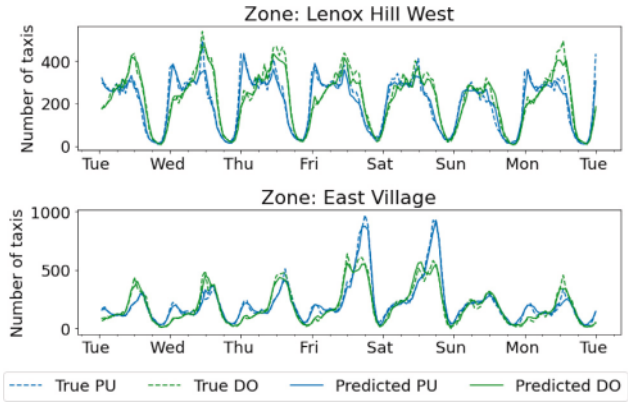

Fig. 3. The figure shows the predictions of dropoffs (DO) and pickups (PU) along with the true pickups and dropoffs for a full week of the test set.

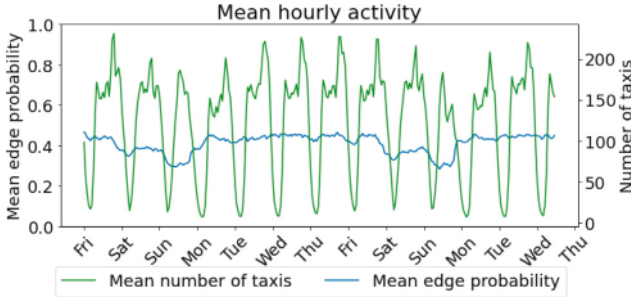

Fig. 4. The mean number of taxi actions (both PUs and DOs) over all zones along with the mean edge probability of the adjacency matrix.

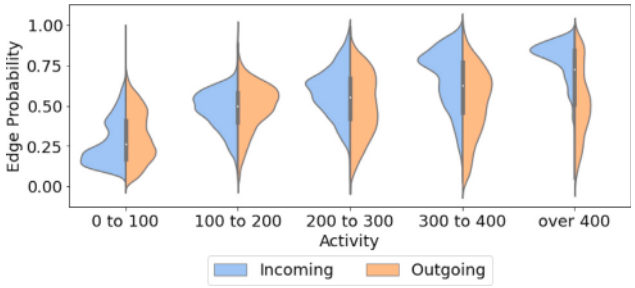

Fig. 5. The distributions of incoming and outgoing edge probabilities for NYC zones grouped by activity levels.

Analyzing the learned adjacency matrix

Now, we look at the best performing NRI model, the NRI DTW, to see what kinds of relationships are put in the adjacency matrix. In Fig. 4, we show the time series of the mean taxi actions per hour of the entire Manhattan, i.e., the mean of pickups and dropoffs for all zones, along with the mean edge probability for the learned adjacency matrix. As can be seen in the figure, the mean edge probability is constant during the weekdays but has apparent dips during the weekend. This indicates that the messages are optimized for predicting weekday trends, and the model thus sends fewer messages during weekends. Looking at Fig. 4, we can also see why this makes sense for the model. First off, there are more weekdays than days on the weekend; secondly, we can see in the figure that, on average, there are fewer taxi trips on weekends than on weekdays. As the model is trained using the loss in Eq. (11), which puts a higher emphasis on large observations, it makes sense for the model to focus on weekdays instead of weekends.

We also analyze which zones send and receive messages. In Fig. 5, we group the zones based on their activity level. We define their activity level as the mean of the pickups and dropoffs. As can be seen in the figure, higher activity in a zone leads to higher incoming edge probabilities. This shows that the model learns to send more messages to zones with high activity. However, from the figure, we can also see that higher activity does not necessarily lead to higher outgoing edge probability. So, the model is not just sending information between high activity zones while ignoring low activity zones but instead sends information from different kinds of zones to the high activity zones. This is, again, a result of the emphasis the loss in Eq. (11) puts on large observations.

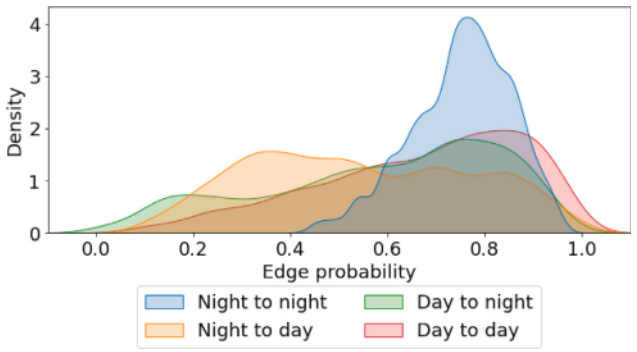

Fig. 6. The distribution of edge probabilities between zones most active during the night and zones most active during the day.

Hence the model learns to create messages that help the most for zones with a significant amount of activity and sends them to them.

For a more detailed look at which zones send to which, we classify the zones as either day zones or night zones depending on when they are the most active. We define day hours to be between 5 am and 6 pm and the rest to be night hours. We then select a subset of zones such that the day and night zones have similar activity to avoid the trends seen in Fig. 5. In Fig. 6, we depict the edge probability distributions for edges between zones of the same and different types. As can be seen in the figure, edges between two night active zones are much more likely than edges going from a night active zone to a day active zone. Similarly, we can see that edges going day active zones are more like than edges from day active zones to night active zones, albeit the difference here is much smaller.

5.4. Experiments on PEMS dataset

In this section, we perform experiments on the PEMS dataset. We again compare the full model against baselines using fixed heuristic-based adjacency matrices and other recent graph-based models from the literature. Furthermore, we compare the best models in the harder case of congested traffic states. Then, we analyze the learned adjacency matrices to see what kind of relation the model finds important. First, we show how a zones connectivity changes during congestion, next we show which other zones a congested zone is connected to, and lastly we show that this behavior is consistent throughout the graph.

Performance on PEMS dataset

First, we compare the full model against baselines with fixed adjacency matrices and a selection of baselines from literature. The different models are:

- **Lag**: The last observed traffic speed is repeated for all predictions.
- **DCRNN**: The Diffusion Convolutional Recurrent Neural Network from Li et al. (2018).
- **STGCN**: The Diffusion Convolutional Recurrent Neural Network from Yu et al. (2018)
- **G-WN**: The Graph WaveNet from Wu et al. (2019)
- **AGCRN**: The Adaptive Graph Convolutional Recurrent Network from Bai et al. (2020)
- **GAGCN**: The Graph Attention based dynamic Graph Convolutional Network from Tang et al. (2020)
- **Fixed empty**: Model with fixed empty adjacency matrix.
- **Fixed full**: Model with fixed full adjacency matrix.
- **Fixed local**: Model with adjacency matrix based on distance between sensors. The distance between sensors is taken from Li et al. (2018).
- **NRI Local prior**: Full NRI model with the local adjacency matrix explained above as prior.

The results can be seen in Table 2. First, we note that our models outperform all of the baselines except AGCRN. AGCRN has the best long term predictions but is still outperformed by NRI for mid and short term predictions. We also note that the Lag model outperforms the DCRNN model for 15- and 30-min predictions, but the DCRNN is better for 60-min predictions. All of the models with fixed adjacency matrices outperform the Lag and DCRNN models, even the model with an empty adjacency matrix. This shows that the recurrent decoder is already a strong prediction model, even without any sharing between nodes. However, the Fixed-local model outperforms the Fixed-empty model showing the benefit of sending information between nodes. Furthermore, the Fixed-full model outperforms the Fixed-local model, showing that the connections in the adjacency matrix based on locality heuristics are not a perfect graph for this problem. The full NRI model achieves similar scores as the Fixed-full model. However, the NRI model has a much more sparse and potentially interpretable adjacency matrix suggesting that the NRI model is able to learn which connections between sensors are important for precise predictions.

Table 2
Results of different models on the PEMS dataset. Results for the DCRNN model is taken from (Li et al., 2018) and the results for Graph WaveNet and STGCN is taken from (Wu et al., 2019).

	15 min			30 min			60 min		
	MAE	RMSE	MAPE	MAE	RMSE	MAPE	MAE	RMSE	MAPE
Lag	1.31	2.80	2.6%	1.65	3.79	3.4%	2.18	5.19	4.7%
DCRNN (Li et al., 2018)	1.38	2.95	2.9%	1.74	3.97	3.9%	2.07	4.74	4.9%
STGCN (Yu et al., 2018)	1.36	2.96	2.9%	1.81	4.27	4.2%	2.49	5.69	5.8%
Graph-WaveNet (Wu et al., 2019)	1.30	2.74	2.7%	1.63	3.70	3.6%	1.95	4.52	4.6%
AGCRN (Bai et al., 2020)	1.16	2.41	2.4%	1.39	3.09	3.0%	1.65	3.80	3.7%
GAGCN (Tang et al., 2020)	1.25	2.47	2.7%	1.59	3.36	3.6%	2.05	4.44	4.8%
Fixed empty	1.05	2.05	2.1%	1.39	3.01	3.0%	1.93	4.31	4.5%
Fixed full	1.02	1.99	2.0%	1.34	2.84	2.8%	1.8	3.90	4.0%
Fixed local	1.04	2.02	2.1%	1.36	2.90	2.9%	1.84	4.07	4.3%
NRI local prior	1.02	2.00	2.0%	1.33	2.84	2.8%	1.79	3.89	4.0%

Table 3
Results of the best performing models on predicting the congested parts of the PEMS dataset.

	15 min			30 min			60 min		
	MAE	RMSE	MAPE	MAE	RMSE	MAPE	MAE	RMSE	MAPE
AGCRN (Bai et al., 2020)	4.92	7.64	66.0%	6.74	10.92	79.3%	9.37	15.17	98.5%
NRI local prior	2.41	3.62	34.6%	4.07	5.92	67.2%	7.76	11.36	132.2%

Moreover, comparing the AGCRNN model and the full NRI model, we can see that the difference in MAE is less than 0.14 mph. This is the MAE over the entire test set which is primarily free flow traffic. It is, however, not during free flow traffic that the performance of a traffic model is most important, as the free flow traffic states are easily predictable for traffic managers. It is instead during congestion that the performance of a traffic model is most important as these states are much harder to predict and have a larger implication for the traffic network. In Table 3 is the performance of the AGCRN model and the full NRI model during states of heavy congestion with speeds below 10 mph. As can be seen in the table, the NRI model mostly outperforms the AGCRN during heavy congestion, especially for short and midterm prediction. The AGCRN, however, does have a better MAPE for long term prediction.

Analyzing the learned adjacency matrix

Next, we look at the time series of mean edge probabilities to see if we can see a correlation between the data and the graph inferred by the encoder. If we look at Fig. 7, we can see the observed traffic speed, the mean incoming and outgoing edge probability for four loop detectors placed along the same freeway in the southbound direction. The sensors placement can be seen in Fig. 14 in the appendix. The first sensor 407 352 is on Freeway 85 just before the merge with traffic from the crossing westbound El Camino Real. The second sensor (407 360) is just after the merge, while the third (407 361) and fourth (402 061) are further south along the freeway. From the plot, we can see that the first and last sensors have regular traffic for the depicted time period. Looking at the mean incoming and outgoing probabilities, we can see that they are noisy but maintain a probability of around 30%. However, looking at the second sensor (407 360), we can see that during the Saturday, there was some congestion lowering the traffic speed. At the beginning of the congestion, we can see that the node first drops and then peaks in outgoing edge probability. After that, we can see that during the congestion, the mean incoming edge probability for the zone is higher, meaning that the node is taking in more information from the rest of the traffic network when predicting the traffic speed at the sensor. Looking at the third zone, we can see that there is a lesser amount of congestion leading to slightly lowered traffic speeds. This then results in slightly raised incoming edge probabilities. This suggests that the model sends more information to congested zones; the more congested, the more information is sent to that zone.

In Fig. 8, the edges with probability higher than 80% for sensor 407 360 at 11:30 on the Saturday in the test set are depicted. At this time, the freeway around the sensor is congested, and the traffic speed is lowered. As can be seen in the figure, the nodes receive information from three different nodes and send information to 2 other nodes. Interestingly, these nodes are not the immediate neighbors of the node. However, the data has a time fidelity of 5 min, and we are predicting the next 12 time steps, i.e., the next hour. Hence the information in the immediate neighbors might be outdated already for the next time step and definitely will be at some point along the prediction window. The nodes the sensor is trying to get information from are all ∼12 min away[5]; hence the information at these sensors might be more important for the full prediction window.

In the top of Fig. 9, the traffic speed and mean incoming and outgoing edge probabilities for sensor 407 360 around 11:30 on Saturday are depicted along with the same information for the sensors that 407 360 sends information to with a probability higher than 80%. As can be seen from the figure, the zones that 407 360 sends information to are zones that have just had some congestion. In the bottom of Fig. 9, the traffic speed and mean incoming and outgoing edge probabilities for sensor 407 360 around 11:30 on

[5] According to the Google Maps API.

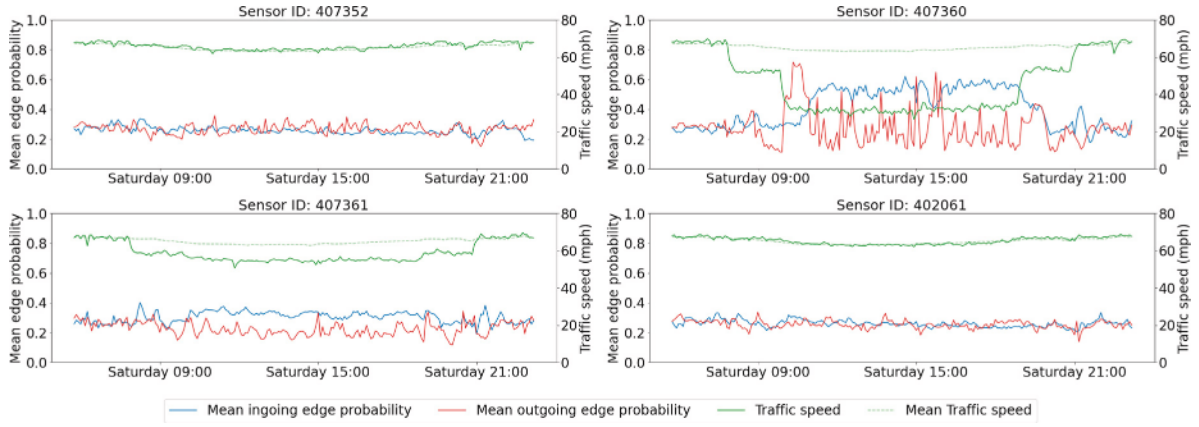

Fig. 7. The mean edge probability for incoming and outgoing edges along with the mean traffic speed and traffic speed at the specific sensor for four sensors along the same freeway. As can be seen, a lower than usual traffic speed at a sensor leads to more incoming edges at the sensor.

Saturday are depicted along with the same information for the sensors that 407 360 receives information from with a probability higher than 80%. As can be seen from the figure, all of the zones that 407 360 receives information from are zones with more congestion than 407 360 itself.

We can then use the inferred latent graphs from the entire test set to see if sending information to congested sensors happens consistently for the entire graph. To do this, we define a sensor as congested if it at a time step has an observed traffic speed that is 30 mph slower than the mean speed of the entire system. This corresponds to the amount of congestion also seen in Fig. 9 and when the Level of Service goes from E to F according to the Highway Capacity Manual (Board, 2000). In Fig. 10, we can see the empirical distribution of edge probabilities going to congested and uncongested sensors. As shown in the figure, the probabilities for edges going to uncongested sensors are centered around 0.26. However, the edge probabilities to congested sensors are much more spread out, with more edges with very low or very high probabilities. From this, we can see that an edge is much more likely to have a high probability if it is going to a congested sensor, i.e., the inferred graph tries to send more information to congested sensors.

In Fig. 11 the empirical distribution of edge probabilities going to congested sensors is split up based on the sensor the edge is coming from. As shown in the figure, edges coming from uncongested sensors are more likely to have a low probability, while edges coming from a congested sensor have a much higher probability of having a high probability. From this, it is clear that the inferred graph prefers to send information to congested sensors from congested sensors. This is even more visible in Fig. 12 where the ratio between edges from congested and uncongested sensors is depicted as a function of edge probability. As can be seen from the figure, the higher the probability, the larger the fraction of edges coming from congested sensors. From this, we can see that the behavior of the graph observed in Fig. 9 is general behavior for the model over the entire test set.

6. Limitations of interpretability

In this section, we will discuss the analysis of inferred graphs as a tool for gaining insights and present which limitations and challenges need to be solved to lead to better interpretability. The assumption behind NRI is that the graph that leads to the best predictive performance is the graph that approximates the true correlations best. Hence it is clear that the formulation of the loss function will have a big impact on the inferred graph. Fig. 5 is a good example of how the inferred graphs follow the inductive bias of the loss function. However, Fig. 6 shows that the inferred graphs can also capture correlations that are not directly a product of the loss function formulation.

The results in Figs. 8 and 9 show that the insights from the inferred graphs can be comprehensible and Figs. 10 to 12 show that the insights are stable. However, it is another question entirely how translucent the insights are. The raw form of the graphs are just a matrix of edge probabilities based on the latent distribution inferred by the encoder for each timestep i.e. $H^{N\times N}$ s.t. $H^{N\times N}_{(i,j)} = \boldsymbol{h}^{(3)}_{(i,j)}$. In this raw form, as seen in Fig. 13 in the appendix, the matrix is not very comprehensible, and as such, some work is required to gain insights based on them. Finding main trends in the graphs, like those presented in Figs. 5 and 7 can be found quite easily following a methodology of looking at how changes in input data change the inferred graphs. The validity of any found trends can then be tested by looking at the distribution over all data points like in Fig. 10, for example. However, finding secondary or weaker trends like in Fig. 6 takes more data wrangling to find. They can be validated in the same manner, though.

7. Conclusion

In this paper, we have presented a graph neural network for spatio-temporal demand prediction in transport problems. The model follows the VAE structure from Neural Relational inference and is extended to take in global features. To the authors' knowledge,

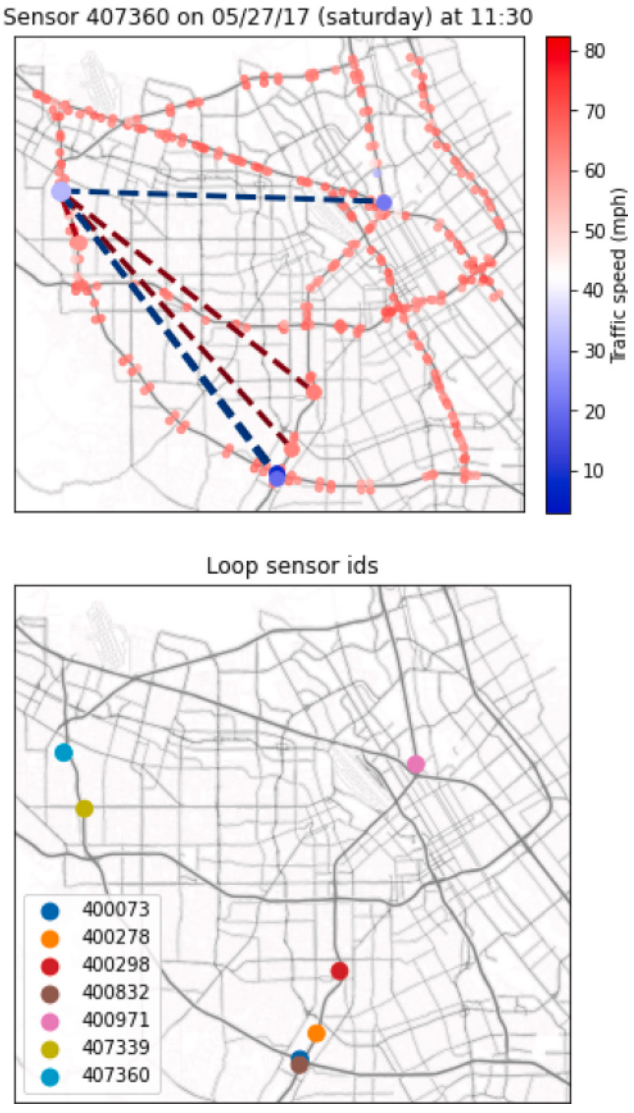

Fig. 8. Top: Learned edges with probability higher than 80% for sensor 407 360 at 11:30 on 05/27/17, which is a Saturday. Blue edge means that the edge is incoming for sensor 407 630, and red means outgoing. Bottom: Sensor ids for the connected sensors. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

this is the first model with a learnable adjacency matrix that is not fixed to a specific number of nodes. Furthermore, it is also the first GNN used for transport problems that is able to utilize a domain prior for the learnable adjacency matrix.

Through experiments on the NYC Yellow Taxicab dataset and the PEMS dataset, we have shown that using a learned, dynamical adjacency matrix can outperform models using fixed adjacency matrices based on common heuristics such as locality and time series correlations. Furthermore, we show that the usage of Bayesian priors on the learned adjacency matrix enables the use of domain knowledge and can increase the model's predictive performance.

By performing a thorough analysis of the learned adjacency matrices, we show that these matrices are interpretable to some extent and demonstrate how they depend on the learning strategy. For example, we see in the case of the NYC Yellow Taxicab dataset that while the adjacency matrix sends information from night active zones to other night active zones the learned adjacency matrix primarily sends messages to the zones with the highest amount of activity. This is because these zones have the highest impact on the negative loglikelihood loss we train the model with. As such, the messages are optimized in aiding predictions in high activity zones, and the adjacency matrix sends these messages to such zones. Another example of the modular interpretability is the case of the PEMS dataset nodes with congestion. They receive information from nodes with more congestion, even if farther away in space and time.

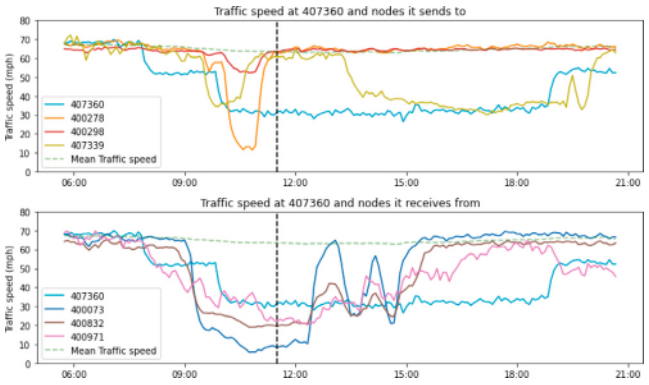

Fig. 9. The traffic speed, mean incoming and mean outgoing edge probabilities at sensor 407 360 and the sensors that 407 360 are connected to with an edge with more than 80% probability. The top figure shows zones connected to via an outgoing edge, and the bottom figure shows zones connected via an incoming edge.

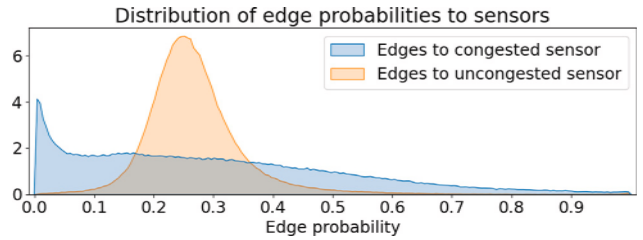

Fig. 10. Empirical distributions of latent edges probabilities going to sensors at congested and uncongested highways.

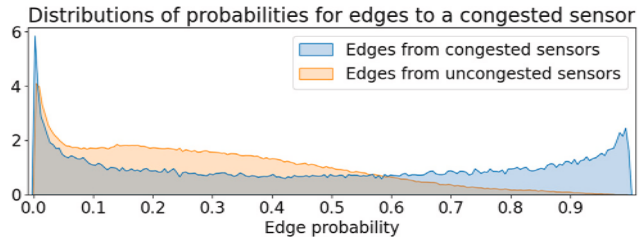

Fig. 11. Empirical distributions of latent edge probabilities going to sensors at congested highways.

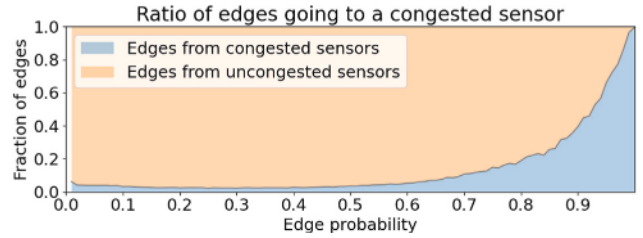

Fig. 12. The ratio edges coming from sensors at congested highways as a function of the edge probability.

We also discussed the limitations and challenges of using analysis of the inferred graphs as a tool for model interpretability. Most notably, we found that the formulation of the model and the loss function can lead to less interesting correlations being the clearest and other correlations requiring more work to uncover.

In future work, we have two directions we would like to pursue. One direction is in extending the model to try and solve some of the limitations brought forward in the last section. Possible extensions could be exchanging the encoder and decoder parts of the model with other GNN based models that might have better inductive biases for the correlations present in the data. Another possible approach is trying to infer a multidimensional distribution of latent edge probabilities, i.e., in essence inferring multiple

Table 4
The number of trainable parameters in the different models used on the NYC dataset.

Model	Number of parameters
CGC	261 880
NRI	255 588
Fixed	12 130

Table 5
The number of trainable parameters in the different models used on the PEMS dataset. Note that only the models with learnable adjacency matrices have been included.

Model	Number of parameters
GAGCN	4 864 069
Graph WaveNet	247 660
AGCRN	659 667
NRI	22 883
Fixed	11 681

graphs. The hypothesis here is that by having multiple graphs, each graph will be able to capture a specific type of correlation, making the subsequent analysis easier.

Another interesting direction is to use the proposed model in a meta learning setup. Since the model is not constrained to a specific number of nodes, the model can be used in setups where it is pre-trained using a big dataset and then fine tuned and evaluated on a smaller dataset.

CRediT authorship contribution statement

Mathias Niemann Tygesen: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing, Visualization. **Francisco Camara Pereira:** Conceptualization, Writing, Supervision. **Filipe Rodrigues:** Conceptualization, Writing, Supervision, Funding acquisition.

Data availability

Data will be made available on request.

Acknowledgment

This project has received funding from the European Union's Horizon 2020 research and innovation programme under grant agreement No 875530.

Appendix A. Experiment details

All models of our models have been trained on an NVIDIA V100 GPU for faster training times. Inference of results has been made on an NVIDIA GeForce RTX 2080 Ti. Training of baselines has also been done on an NVIDIA GeForce RTX 2080 Ti due to limitations in baselines source code. Below we have hyperparameters for our models as well as memory and inference time comparison between our models and the most comparable baseline.

A.1. Hyperparameters

All hyperparameters can be found at https://github.com/MathiasNT/NRI_for_Transport.

A.2. Model size and inference time comparisons

In Table 4 the number of trainable parameters for the models used on the NYC dataset can be seen. As seen from the table, the NRI and CGC are of comparable sizes. Note that the Fixed model only uses the recurrent GNN decoder and does not learn an adjacency matrix.

In Table 5 the number of parameters of the models used on the PEMS dataset can be seen. Only the baselines with learnable adjacency matrices can be seen. The baseline models' hyperparameters are based on the hyperparameters from the respective papers with some finetuning to the dataset. As can be seen from the table the NRI uses fewer parameters than the comparable models. Note that the Fixed model only uses the recurrent GNN decoder and does not learn an adjacency matrix.

On the PEMS data, we also compared the inference time of the NRI model to the inference time of the GAGCN model. We note that this is the most comparable baseline as it is the only other model with a dynamic adjacency matrix. We ran inference over the test set and computed the average time it took for the models to make a prediction of the next hour based on the previous hour. On average the NRI model took 0.832 s while the GAGCN took 0.810 s. Since the data, and hence the predictions, are in five minute intervals both of these inference times are plenty fast to be used in a production setting.

Appendix B. Result figures

B.1. Raw adjacency matrix

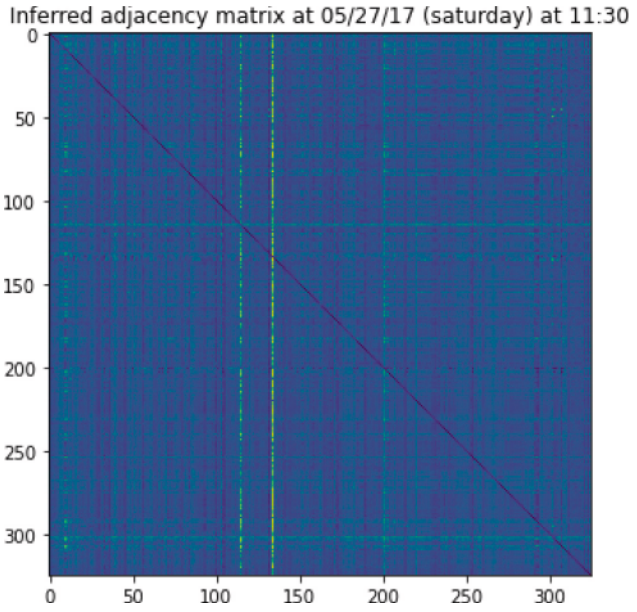

Fig. 13. The raw inferred adjacency matrix using the same model and timestep as Fig. 8. Each entry in the adjacency matrix is the edge probability inferred by the NRI encoder.

B.2. Position of sensors in Fig. 14

Fig. 14. The mean edge probability for incoming and outgoing edges along with the mean traffic speed and traffic speed at the specific sensor for four sensors along the same freeway. As can be seen, a lower than usual traffic speed at a sensor leads to more incoming edges at the sensor.

B.3. PEMS inferred graph example two

In this section be present another result from the analysis of the PEMS model. The result is similar to that shown in Figs. 8 and 9. In Fig. 15 we depict the sensor 400 863 along with the edges with higher than 80% probability on 05/26/17 at 5:10. As seen in the plot, the sensor only has incoming edges with high probability, and all of these edges are coming from sensors along the same freeway. This shows quite different behavior from Fig. 8. In Fig. 16, the data at the connected sensors from Fig. 15 is depicted. From the figure it is clear that we are also in a quite different scenario than that around Fig. 8 as all of the connected sensors show higher than average speeds at the time.

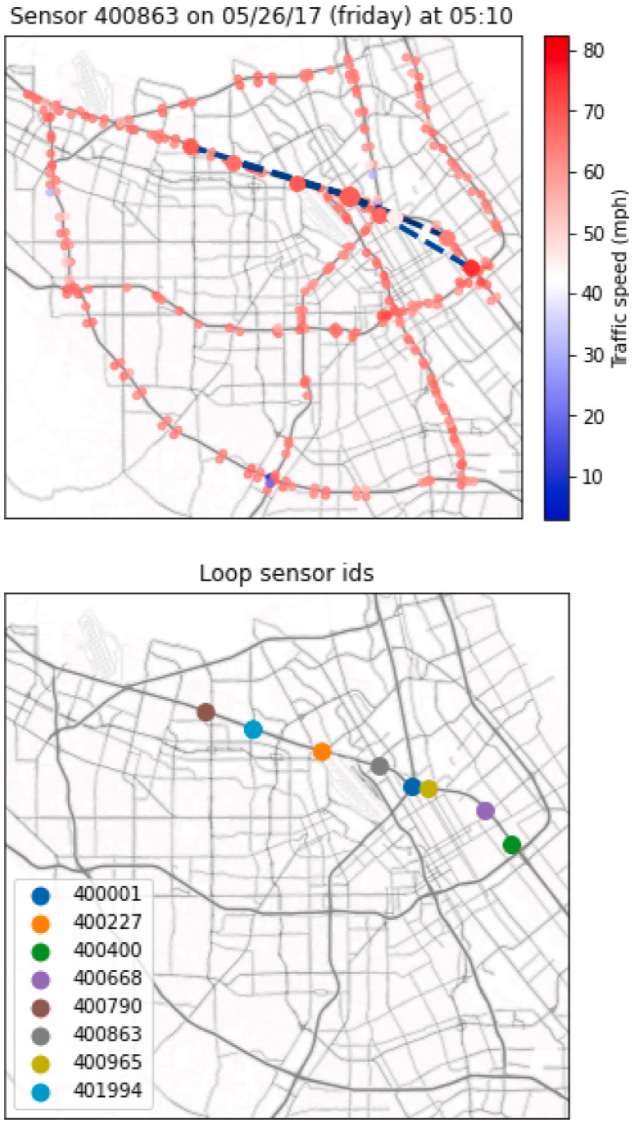

Fig. 15. Top: Learned edges with probability higher than 80% for sensor 400 853 at 5:10 on 05/26/17. Blue edge means that the edge is incoming for sensor 400 853. Bottom: Sensor ids for the connected sensors. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

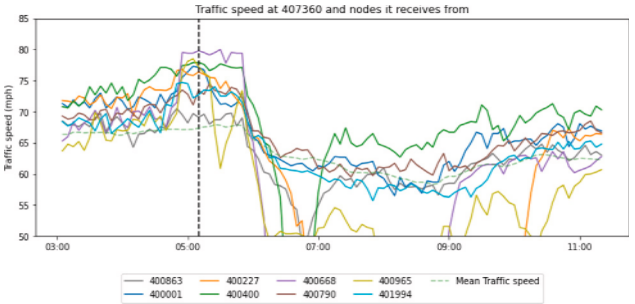

Fig. 16. The traffic speed and mean incoming and outgoing edge probabilities at senor 400 853 and the sensors that 400 853 is connected to with a incoming edge with more than 80% probability.

References

Bai, L., Yao, L., Li, C., Wang, X., Wang, C., 2020. Adaptive graph convolutional recurrent network for traffic forecasting. ArXiv, abs/2007.02842.

Barcelo-Vidal, C., Aguilar, L., Martín-Fernández, J.A., 2011. Compositional VARIMA Time Series. John Wiley & Sons, Chichester.

Battaglia, P.W., Hamrick, J.B., Bapst, V., Sanchez-Gonzalez, A., Zambaldi, V.F., Malinowski, M., Tacchetti, A., Raposo, D., Santoro, A., Faulkner, R., Gülçehre, Ç., Song, H.F., Ballard, A.J., Gilmer, J., Dahl, G.E., Vaswani, A., Allen, K.R., Nash, C., Langston, V., Dyer, C., Heess, N., Wierstra, D., Kohli, P., Botvinick, M., Vinyals, O., Li, Y., Pascanu, R., 2018. Relational inductive biases, deep learning, and graph networks. CoRR, abs/1806.01261.

Board, T.R., 2000. Highway Capacity Manual. Transportation Research Board (TRB), ISBN: 978-0-309-06681-5, 13–10.

Bronstein, M.M., Bruna, J., Cohen, T., Velickovic, P., 2021. Geometric deep learning: Grids, groups, graphs, geodesics, and gauges. CoRR, abs/2104.13478.

Diao, Z., Zhang, D., Wang, X., Xie, K., He, S., Lu, X., Li, Y., 2019. A Hybrid Model for Short-Term Traffic Volume Prediction in Massive Transportation Systems. IEEE Trans. Intell. Transp. Syst..

Gilmer, J., Schoenholz, S.S., Riley, P.F., Vinyals, O., Dahl, G.E., 2017. Neural message passing for quantum chemistry. CoRR, abs/1704.01212.

Hamilton, W.L., Ying, R., Leskovec, J., 2017. Inductive representation learning on large graphs. CoRR, abs/1706.02216.

Jang, E., Gu, S., Poole, B., 2017. Categorical reparameterization with gumbel-softmax. In: 5th International Conference on Learning Representations, ICLR 2017 - Conference Track Proceedings.

Jiang, W., Luo, J., 2021. Graph neural network for traffic forecasting: A survey. arXiv:2101.11174.

Jin, G., Cui, Y., Zeng, L., Tang, H., Feng, Y., Huang, J., 2020. Urban ride-hailing demand prediction with multiple spatio-temporal information fusion network. Transp. Res. C 117, 102665.

Kingma, D.P., Welling, M., 2014. Auto-encoding variational bayes. In: 2nd International Conference on Learning Representations, ICLR 2014 - Conference Track Proceedings.

Kipf, T.N., Fetaya, E., Wang, K., Welling, M., Zemel, R.S., 2018. Neural relational inference for interacting systems. In: Proceedings of the 35th International Conference on Machine Learning, ICML 2018, Stockholmsmässan, Stockholm, Sweden, July 10-15, 2018. In: Proceedings of Machine Learning Research, vol. 80, PMLR, pp. 2693–2702.

Kipf, T.N., Welling, M., 2016. Semi-supervised classification with graph convolutional networks. CoRR, abs/1609.02907.

Li, Y., Yu, R., Shahabi, C., Liu, Y., 2018. Diffusion convolutional recurrent neural network: Data-driven traffic forecasting. arXiv: Learning.

Liu, H., Ong, Y.-S., Shen, X., Cai, J., 2020. When Gaussian process meets big data: A review of scalable GPs. IEEE Trans. Neural Netw. Learn. Syst. 31 (11), 4405–4423.

Maddison, C.J., Mnih, A., Teh, Y.W., 2017. The concrete distribution: A continuous relaxation of discrete random variables. In: 5th International Conference on Learning Representations, ICLR 2017 - Conference Track Proceedings.

Molnar, C., 2022. Interpretable Machine Learning, second ed..

Rodriguez-Deniz, H., Jenelius, E., Villani, M., 2017. Urban network travel time prediction via online multi-output Gaussian process regression. In: 2017 IEEE 20th International Conference on Intelligent Transportation Systems. ITSC, IEEE, pp. 1–6.

Tang, C., Sun, J., Sun, Y., Peng, M., Gan, N., 2020. A general traffic flow prediction approach based on spatial-temporal graph attention. IEEE Access 8, 153731–153741.

Williams, B.M., Hoel, L.A., 2003. Modeling and forecasting vehicular traffic flow as a seasonal ARIMA process: Theoretical basis and empirical results. J. Transp. Eng..

Wu, Z., Pan, S., Long, G., Jiang, J., Zhang, C., 2019. Graph WaveNet for deep spatial-temporal graph modeling. In: IJCAI.

Xu, J., Rahmatizadeh, R., Bölöni, L., Turgut, D., 2018. Real-time prediction of taxi demand using recurrent neural networks. IEEE Trans. Intell. Transp. Syst. 19, 2572–2581.

Ye, J., Sun, L., Du, B., Fu, Y., Xiong, H., 2021. Coupled layer-wise graph convolution for transportation demand prediction. In: AAAI.

Yin, X., Wu, G., Wei, J., Shen, Y., Qi, H., Yin, B., 2021. Deep Learning on Traffic Prediction: Methods, Analysis and Future Directions. IEEE Trans. Intell. Transp. Syst..

Yu, T., Yin, H., Zhu, Z., 2018. Spatio-temporal graph convolutional networks: A deep learning framework for traffic forecasting. In: IJCAI.

Zhang, Z., Li, M., Lin, X., Wang, Y., He, F., 2019. Multistep speed prediction on traffic networks: A deep learning approach considering spatio-temporal dependencies. Transp. Res. C.

Zhang, J., Zheng, Y., Qi, D., Li, R., Yi, X., 2016. DNN-based prediction model for spatio-temporal data. In: Proceedings of the 24th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems.